Decomposing the Score of an NVARC-Derived Pipeline on ARC-AGI-2: Generation, Selection, and Run-to-Run Noise

Md. Hamid Hosen
Dept. of CSE, East Delta University
mdhamidhosen4@gmail.com

MD. Foysal Emon Shanto
Dept. of CSE, Jashore University of Science and Technology
foyshalsk348@gmail.com

Md Esfer Abdus Sami
Dept. of CSE, North South University
esfer.sami.me@gmail.com

Team Hack2Publish, ARC Prize 2026

Abstract

NVARC, the winning ARC Prize 2025 entry, has three components. We study one of them, its Qwen3-4B solver derived from the ARChitects, as modified in 34 notebooks our team ran for ARC Prize 2026: per-task LoRA test-time training, a depth-first search that keeps outputs with probability above 0.2 in 16 augmented views per test input, augmentation-based rescoring and two attempts. On the public ARC-AGI-2 evaluation set (120 tasks, 172 test inputs) we re-score 41 distinct results of our NVARC runs and decompose two full runs whose logs give, for every test input, the rank of the answer among the decoded candidates. (i) *Generation*: at these settings, a correct grid is decoded for 56 and 55 of 172 test inputs; 116 and 117 get no correct candidate above the threshold. (ii) *Selection*: perfect selection would raise the task-normalised score from 35.83 to 40.50 and from 33.50 to 41.17 of 120 (+4.67 and +7.67 points, or +13.0 % and +22.9 % of the score). (iii) *Candidates*: a test input gets 14.0–14.4 candidates but 7.3–7.6 distinct grids. For 47 and 48 inputs without a correct candidate, mean cell agreement exceeds 90 %, a bar that copying the test input clears for 26 and 27 of them. (iv) *Noise*: three runs of one program with identical seeds answer 49, 45 and 51 test inputs (35.83, 33.50, 37.67); 41 are shared and 57 answered by at least one. Coverage grows across runs: 55–56 test inputs in one pool, 61 in two pools, 70 in all our submissions and pools together. We also document notebook diagnostics that misreport these quantities. The model's synthetic training data were in part derived from descriptions of these evaluation tasks, so coverage here may not transfer to unseen tasks.

1 Introduction

ARC-AGI-2 [7] continues the Abstraction and Reasoning Corpus [5]. Each task gives a few input–output grid pairs, and a solver must produce the output grid for one or more new test inputs. In the Kaggle competitions of the ARC Prize [6, 8, 9], a solver runs offline under a fixed compute budget. It may submit two attempts per test input, and a test input counts as solved if either attempt matches the answer exactly.

The winning entry of 2025, NVARC [16], has three components: synthetic data generation, an improved version of the ARChitects solution [10], and an improved Tiny Recursive Model [13]. The ARChitects-derived component fine-tunes a Qwen3-4B model [15] on grids and combines it with test-time training [1], a thresholded depth-first search and augmentation-based candidate scoring. The search and the rescoring follow the ARChitects [10]; NVARC contributed a batched implementation of the search. NVARC report 27.64 % on the 2025 public leaderboard, 24.0 % on the private leaderboard and 29.72 % for a post-deadline variant [16]. Its code and model are public, and our team ran or adapted 34 Kaggle notebooks built on the Qwen3-4B component for ARC Prize 2026. We study that component only, as modified in our notebooks. We did not run the Tiny Recursive Model; NVARC report that it gave no gain on top of the 4B model (a score of 27.22 with and without it).

Improving such a pipeline requires knowing which stage limits it. A test input is lost either because no correct grid is generated or because a correct grid is generated but not among the two submitted. These two failure modes call for different work: better *selection* (rankers, voting, verification) or better *generation* (search, sampling, model). Several of our notebook versions changed the selection logic (a "dual" selector, three alternative selectors). In this paper we measure how much headroom selection has at the settings we ran.

We decompose pass@2, as used in Chen et al. [4], into *pool coverage* (is a correct grid among the decoded candidates?) and *selection* (is it among the two submitted?). We measure both on two complete runs whose logs report, for every test input, where the answer ranks among the candidates. We complement this with all the results we could retrieve from our NVARC notebooks (41 distinct results), including a third run of the same program. Our contributions are:

- a decomposition of the pass@2 score of this NVARC-derived pipeline into pool coverage and selection on two full runs of the public ARC-AGI-2 evaluation set. At the shared settings (threshold 0.2, 16 views), the pool covers 56 and 55 of 172 test inputs, and perfect selection would add +4.67 and +7.67 task-normalised points of 120, or 13.0 % and 22.9 % of the score (Section 4.1);
- a description of the test inputs that receive no correct candidate above the threshold (67.4 % and 68.0 %): how their candidates compare with the answer and with a copy-the-input baseline, and how many distinct grids the search returns (Sections 4.2 and 4.3);
- a measurement of run-to-run variation from three runs of one program with identical seeds (45–51 correct test inputs), and oracle unions over runs and code variants (Section 4.4);
- a list of measurement pitfalls in the notebooks' own diagnostics, with the corrected numbers (Section 5).

We report what we measured: one model with one set of generation settings, run in several code versions. We do not claim that no variant of the pipeline can do better. Section 7 lists the threats to validity and the variants without pool data.

2 The NVARC-derived pipeline as run in our notebooks

All six full runs in this study use the same model, the same test-time training recipe and the same search settings (threshold 0.2, 16 views); a few other notebooks changed these settings (Section 7). Settings below are taken from the solver source saved with run R2 (`arc_solver.py`, `arc_loader.py`, `arc_decoder.py`). Run R1 uses the same code except for one constant, noted below.

Our notebooks modify the released solver. Compared with our team's reproduction notebook of the 2025 release, the main differences are:

- the probmul ranker uses a per-grid baseline instead of a fixed baseline of 3;
- the submitted pair comes from the "dual" selector instead of the kgmon top 2;
- the per-task cap is 1600 s instead of 1200 s;
- augmentation seeds are derived from MD5 hashes instead of Python's randomised `hash()`;
- a placeholder grid is written when no output is produced.

Run A still uses the fixed baseline, the 1200 s cap and `hash()` seeds. Run B still derives its scoring-augmentation seed from `hash()`. The results below describe the pipeline as run in these notebooks. Figure 1 shows the architecture of our study. Panel (a) is the solver, with the stage at which each of our measurements applies. Panel (b) shows how the numbers are produced from the saved notebook outputs.

Model and test-time training. The base model is `sorokin/qwen3_4b_grids15_sft139`, a model that NVARC fine-tuned on grid tasks from Qwen3-4B-Thinking-2507 [15]. It is loaded in bfloat16 with Unsloth [17] (version 2025.9.7 in the logs), installed from a patched build (`sorokin/pip-install-unsloth-flash-patch`) whose inference path calls flash attention. For every task, a fresh LoRA adapter [12] (rank 256, $\alpha = 32$, rank-stabilised scaling [14]) is trained for one epoch on all attention and MLP projections; the embeddings and the output head are listed as targets, and Unsloth trains them directly as full copies (`modules_to_save`) rather than through LoRA. The training data are augmented copies of the task's demonstration pairs: 8 dihedral transforms, each with 16 independently drawn permutations of all ten colours, and with the order of the demonstration pairs shuffled. This gives 128 training samples, and training runs 128 steps. Training uses learning rate $5 \cdot 10^{-5}$ with cosine decay, batch size 1 and a maximum sequence length of 8192 tokens.

Generation. Each test input is presented in up to 16 augmented views (8 dihedral transforms, each with 2 independently drawn colour permutations, and shuffled demonstration order). For each view, a depth-first search (`turbo_dfs`) keeps a partial output only while its cumulative negative log-likelihood stays below $-\ln 0.2 \approx 1.609$. A completed output is therefore one whose probability under the adapted model, given that view, exceeds 0.2. Let Y_v be the completed outputs of view v and C_u the multiset of candidates of test input u. Distinct completed outputs are prefix-free, so their probabilities sum to at most 1:

$$1 \;\geq\; \sum_{y \in Y_v} p_\theta(y \mid v) \;>\; 0.2\,|Y_v| \quad\Longrightarrow\quad |Y_v| \leq 4, \qquad |C_u| \leq 16 \cdot 4 = 64. \tag{1}$$

The code also caps the number of next tokens expanded per step: 10 in R2 and 5 in the other full runs. This cap never binds, because a token survives only if its conditional probability exceeds 0.2,

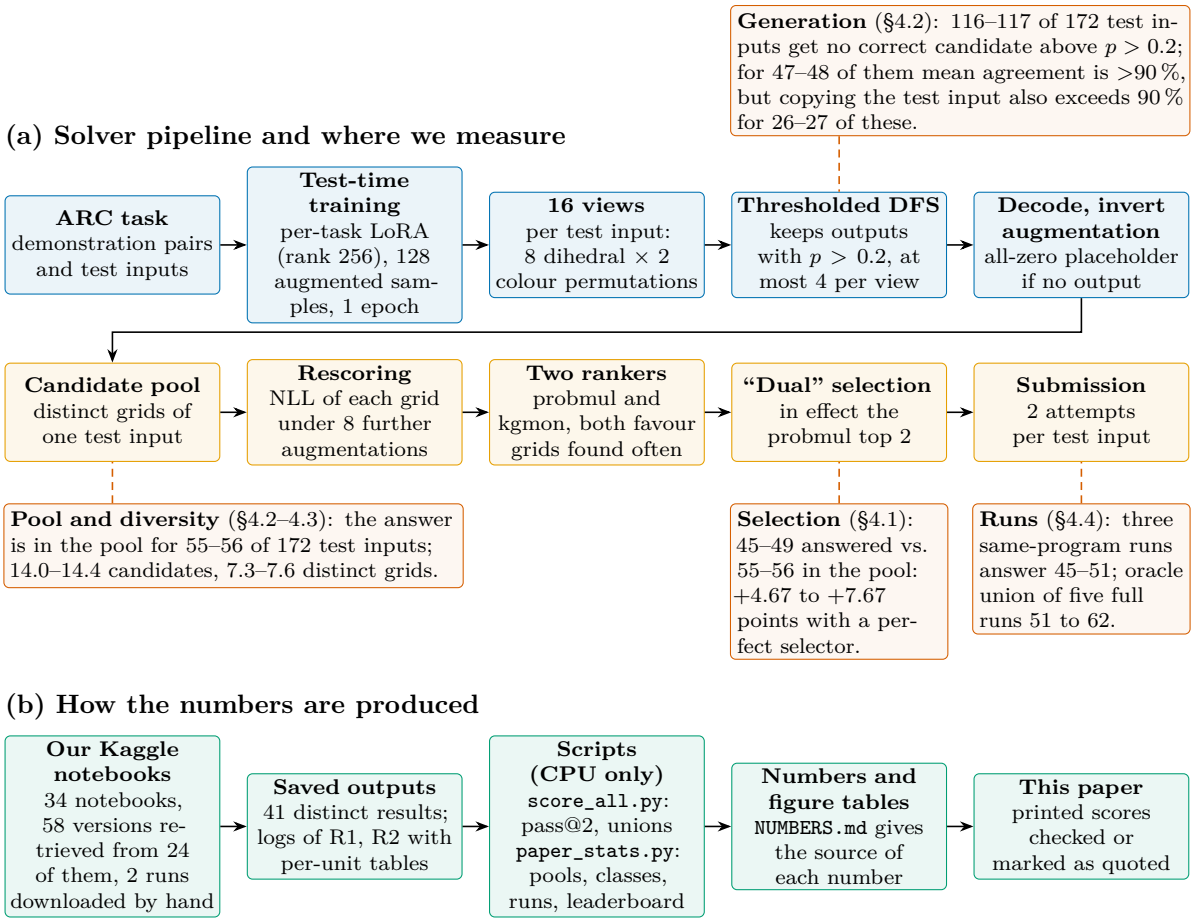


Figure 1: Architecture of the study. (a) The NVARC-derived solver as run in our notebooks (blue: generation; orange: scoring and selection). Red boxes give the quantity measured at each stage, as ranges over the two logged runs R1 and R2 (more runs for the last box). (b) The measurement pipeline: scores and statistics are recomputed from saved outputs by two scripts that need no GPU; the few values that cannot be recomputed are quoted from run logs and marked as such.

and at most 4 tokens can. Each output is decoded into a grid and mapped back through the inverse augmentation. If the search returns no completed output for a test input, the pipeline writes an all-zero placeholder grid of the test-input shape.

Scoring. Every distinct grid y is re-scored by the adapted model under 8 further augmentations. Each occurrence g of y (from different views or search paths) carries its search NLL b_g and eight augmentation NLLs $a_{g,1..8}$. Two rankers order the distinct grids by

$$\text{probmul}(y) = \sum_{g \in G(y)} (\beta - b_g) + \frac{1}{|G(y)|} \sum_{g \in G(y)} \sum_{j=1}^{8} (\beta - a_{g,j}), \qquad \beta = \max_{g \in G(y)} b_g + 1, \tag{2}$$

$$\text{kgmon}(y) = |G(y)| - \frac{1}{|G(y)|} \sum_{g \in G(y)} \frac{1}{8} \sum_{j=1}^{8} a_{g,j}, \tag{3}$$

where $G(y)$ is the set of occurrences of y (functions `score_full_probmul_3` and `score_kgmon`). The augmentation NLLs are computed once per grid, so $a_{g,j}$ is the same for every occurrence of y.

Algorithm 1 The NVARC-derived solver for one task, as run in our notebooks (code of R2). In the code, the batches of different test inputs of a task are interleaved.

Require: task t with demonstration pairs D_t and test inputs $x_1, \dots, x_{n_t}$; base model θ_0; threshold $\tau = -\ln 0.2$

$\theta \leftarrow \theta_0$ plus a fresh LoRA adapter, trained one epoch on $\mathrm{Aug}(D_t)$ ▷ 8 dihedral × 16 colour perm.
for $i = 1, \dots, n_t$ **do**
  $C_i \leftarrow \emptyset$ ▷ multiset of candidates $(y, b, \mathbf{a})$
  for each batch of 4 of the 16 views v of x_i **do** ▷ 8 dihedral × 2 colour perm.
    if task t has used more than 1600 s **then break**
    end if
    for each completed output $(\sigma, b) \in \textsc{ThresholdedDFS}(\theta, v(D_t, x_i), \tau)$ **do** ▷ Algorithm 2
      $g \leftarrow \mathrm{decode}(\sigma)$; **if** g is not a valid grid **then continue**
      $y \leftarrow v^{-1}(g)$ ▷ undo the augmentation
      if y has no augmentation scores yet **then**
        $\mathbf{a}_y \leftarrow \left(\mathrm{NLL}_\theta(w_j(y) \mid w_j(D_t, x_i))\right)_{j=1}^{8}$ ▷ 8 further augmentations w_j
      end if
      $C_i \leftarrow C_i \cup \{(y, b, \mathbf{a}_y)\}$
    end for
  end for
  if $C_i = \emptyset$ **then** $C_i \leftarrow \{(\mathbf{0}_{\mathrm{shape}(x_i)},\ 999,\ 999 \cdot \mathbf{1})\}$ ▷ placeholder
  end if
  $L_a \leftarrow$ distinct grids of C_i by decreasing probmul (2); $L_b \leftarrow$ the same by kgmon (3)
  $L \leftarrow \mathrm{dedup}(L_a[1..3] \,\|\, L_b[1..3])$
  submit $(L[1], L[2])$, or $(L[1], L[1])$ if $|L| = 1$ ▷ $L[2] = L_a[2]$ whenever $|L_a| \geq 2$
end for

Both rankers reward grids found many times. Because the baseline β is set per grid, a grid found once scores $9 + 8b - \sum_j a_j$: its search NLL b enters with a positive weight, and the scores of different grids are not on a common scale.

Selection. The submitted pair ("dual selection") is built from the top 3 grids of each ranker, de-duplicated with the probmul list first. Attempt 1 is the probmul top grid, and attempt 2 is the next grid in that pooled list. Both rankers order the same set of distinct grids. Attempt 2 is therefore the probmul rank-2 grid whenever there are two or more grids, and the kgmon ranking never changes the submitted pair. In both logged runs, the set of correctly answered test inputs equals the set whose answer probmul ranks 1 or 2 (Section 4.1). Algorithms 1 and 2 give the solver and the search as pseudocode, following the code.

Compute. Runs use a Kaggle machine with four NVIDIA L4 GPUs (one solver worker per GPU) and a 12-hour limit. A search call stops after 540 s. Before each batch of 4 views, generation for a task stops if the task has already used 1600 s in total, training included. A full run over all 120 evaluation tasks took 4.68–4.80 h (Table 1). The hidden-set rerun has 240 tasks; at the same time per task it would take about 9.4 h of the 12 h. This is a projection, not a measurement. In R1 and R2 no task timed out (Section 4.2), so on the evaluation set the time budget was not the binding limit; the number of candidates is set by the threshold and the number of views.

Algorithm 2 Thresholded depth-first search (`turbo_dfs`) for one view. In the code, the views of a batch are searched together, and a call stops after 540 s. K is 10 in R2 and 5 in the other full runs; it never binds (Section 2).

function THRESHOLDEDDFS(θ, prompt h, τ)
  return EXPAND(h, 0, $m_{\max}$) ▷ each returned (σ, s) has $p_\theta(\sigma \mid h) = e^{-s} > e^{-\tau} = 0.2$
end function
function EXPAND(h, s, m) ▷ s: NLL of the partial output, m: tokens left
  $O \leftarrow \emptyset$
  for each grid token w with $s_w = s - \log p_\theta(w \mid h) < \tau$, by increasing s_w **do**
    if $w =$ EOS **then**
      $O \leftarrow O \cup \{(\text{EOS}, s_w)\}$
    else if $m > 1$ and w is among the K best non-EOS tokens **then**
      $O \leftarrow O \cup \{(w \| \sigma, s') : (\sigma, s') \in \text{EXPAND}(h \| w, s_w, m-1)\}$
    end if
  end for
  return O
end function

3 Data and measurement

Evaluation set and metric. We use the public ARC-AGI-2 evaluation set [2, 7]: 120 tasks with 172 test inputs. We call one test input a *unit*. A unit is correct if attempt 1 or attempt 2 equals the answer grid (pass@2). The *task-normalised score* gives each correct unit the weight $1/n_t$, where n_t is the number of test inputs of its task, and sums over units. Its maximum is 120. For unit u of task $t(u)$, with answer $y_u^\star$ and submitted attempts $\hat{y}_u^1, \hat{y}_u^2$,

$$c_u = \mathbf{1}\left[\, y_u^\star \in \{\hat{y}_u^1, \hat{y}_u^2\} \,\right], \qquad S = \sum_u \frac{c_u}{n_{t(u)}} \in [0, 120]. \tag{4}$$

The competition overview [8] words the leaderboard score as an average over test outputs. The leaderboard values in our records fit per-task averaging instead: all 18 distinct per-notebook minimum and maximum values are multiples of 100/720 to two decimals, as per-task averaging over 120 tasks with one to three test inputs produces, while no plain average over N test outputs ($100 \le N < 400$) fits them. This relies on the public leaderboard having 120 tasks, like the public evaluation set (it is the public half of the 240 hidden tasks). We therefore read the leaderboard as the task-normalised score expressed as a percentage. We report both units (of 172) and the task-normalised score (of 120); where we compare with the leaderboard, we also give the score as a percentage of 120. Scores are recomputed from the saved submission files against the public answers. For the two downloaded runs, no submission file was saved; we use the set of correct units printed by the decoder's built-in pass@2 benchmark, whose code compares the first two submitted grids with the answer.

Overlap with the model's training data. NVARC's synthetic training data were built in part from descriptions of the public evaluation tasks: the authors labelled 29 evaluation tasks by hand and generated descriptions of the remaining 91, and their write-up states that these descriptions were "leaked" into the synthetic data (sections 2.1 and 3.5 and Table 1 of Sorokin and Puget [16]; the subset built from both training and evaluation tasks is 54.9 % of the samples). NVARC's training script for `qwen3_4b_grids15_sft139` runs 12,716 steps at batch size 256, which matches the size

of that mixture. The link to our model is inferred from this step count and is not confirmed. Every pool, rank and closeness figure in this paper is therefore measured on tasks whose descriptions may have shaped the training data, and may overstate behaviour on unseen tasks. As a mitigating check, our six full runs score 27.92–31.53 % on the evaluation set, and ten leaderboard submissions of one notebook version average 29.36 % on hidden tasks (Section 4.4). We did not measure the size of this effect.

Runs. Our team's Kaggle account holds 34 notebooks that use the NVARC solver. For 24 of them the Kaggle API returned saved outputs, 58 versions in all. Their outputs contain 39 distinct submission files (by SHA-256) from 20 of these notebooks. Two more runs (B and C in Table 1) were downloaded by hand as notebooks with their cell outputs; B's notebook is not among the 34. That makes 41 distinct results of our NVARC runs, most of them partial runs over a subset of tasks. Six runs trained and searched all 120 tasks (Table 1). A, B and D are versions of the pipeline with code changes (fixes, selector variants, diagnostics). R1, R2 and C are three runs of one program. R1 and R2 are versions 1 and 2 of one notebook, and C is a third run of the same code, from a copy of that notebook. Their solver, loader and decoder code is identical, except that R2 uses the non-binding cap of 10 above. Their 240 seed lines (one training and one evaluation seed per task) are identical. Yet their training losses differ (e.g. 0.018519, 0.018989 and 0.018372 on task `271d71e2` in R1, R2 and C), and so do their results. Seeds do not fix outcomes on this stack: NVARC report that the batched search is numerically non-deterministic [16], as batched inference generally is [11]. We did not isolate the source, and we cannot show that the three runs used the same Kaggle image or GPU assignment. We use these runs to measure run-to-run variation (Section 4.4). R1 and R2 end with analysis cells that print, for every unit, the rank of the answer among the decoded candidates under both rankers, the number of candidates and distinct grids, and the mean cell agreement of the candidates with the answer. The candidate grids themselves were not saved. We parse these tables from the run logs and use our own counts. The "candidate pool" P_u of a unit is the set of distinct grids the run decoded for it at the threshold used (the distinct elements of C_u). A unit is *in the pool* if one of them equals the answer. An answer that the adapted model rates at $p \le 0.2$ in every view is not in the pool, so "no correct candidate" is always conditional on this threshold. *Oracle selection* submits the answer for every in-pool unit; its score is the pool coverage O. With r_u the probmul rank of $y_u^\star$ in P_u ($r_u = \infty$ if it is absent),

$$O = \sum_u \frac{\mathbf{1}[\, y_u^\star \in P_u \,]}{n_{t(u)}}, \qquad H = O - S, \qquad C(k) = \big|\{\, u : r_u \le k \,\}\big|. \tag{5}$$

H is the selection headroom and $C(k)$ the top-k coverage. The submitted grids belong to P_u, so $S \le O$, and every unit falls into exactly one of three groups: correct, in the pool but not submitted, or not in the pool. Algorithm 3 gives the decomposition as pseudocode.

Leaderboard records. Public-leaderboard scores come from our team's submission list as of 2026-10-19, matched by notebook: 63 scored submissions (one of them 0.00) from 19 notebooks. Submissions uploaded as a raw file are not included. The public leaderboard is computed on the 120-task public half of the hidden tasks. Each submission is a fresh run of the notebook. Leaderboard scores are percentages, not points out of 120.

Reproducibility. `scripts/score_all.py` re-scores every submission and computes the unions. `scripts/paper_stats.py` computes the pool, generation, diversity, union and leaderboard statistics and the figure data from the saved logs and submissions. Neither script needs a GPU. The few

Algorithm 3 Pool decomposition of one logged run (`scripts/paper_stats.py`).

Require: per-unit table of the run (r_u, $|C_u|$, $|P_u|$, a_u, placeholder flag); the run's submission; the answers $y^\star$

for each of the 172 units u **do**
  if $c_u = 1$ **then**
    class(u) $\leftarrow$ correct ▷ Equation (4)
  else if $r_u < \infty$ **then**
    class(u) $\leftarrow$ in the pool, not submitted
  else if u has only the placeholder grid **then**
    class(u) $\leftarrow$ no completed output
  else if $a_u > 90$ **then**
    class(u) $\leftarrow$ close ▷ Equation (6)
  else if $a_u > 50$ **then**
    class(u) $\leftarrow$ partial
  else
    class(u) $\leftarrow$ far
  end if
end for
compute S, O, H and $C(k)$ (Equations (4) and (5)); assert $\{u : c_u = 1\} \subseteq \{u : r_u < \infty\}$
report whether $\{u : c_u = 1\} = \{u : r_u \leq 2\}$, and the duplicate ratios d_u (Equation (7))
over all full runs: union growth U_m (Equation (8)) for every order of the runs

numbers quoted directly from a run log are marked as printed by the run. `paper/NUMBERS.md` lists the source of every number. The analysis is reproducible from the saved outputs; the GPU runs themselves are not, since the candidate grids were not saved and the search is not deterministic.

4 Results

4.1 Selection headroom: 13–23 % of the score

Table 3 decomposes the two logged runs. In R1, the submission answers 49 units (35.83 of 120, 29.86 %) and the pool covers 56 (40.50, 33.75 %): a headroom of +4.67 points (7 units), or 13.0 % of the score. In R2, the submission answers 45 units (33.50, 27.92 %). A correct grid is present for 55 units, so oracle selection would reach 41.17 (34.31 %): a headroom of +7.67 points (10 units), or 22.9 % of the score. Of the in-pool answers, 7 of 56 (R1) and 10 of 55 (R2) are lost in selection. The rest of the gap to a perfect score is not a selection problem at these settings: 116 units in R1 and 117 in R2 have no correct candidate above the threshold. Table 2 gives task-clustered bootstrap intervals. The headroom intervals exclude zero in both runs, and the range of the two headroom values (+4.67 to +7.67) rests on two runs of one program; its width, 3.00 points, is itself run-to-run variation (Section 4.4). Runs B and C print task-level pool tables but no per-unit statistics. In them the answer is in the pool for 47 and 48 tasks (as printed by the runs), against 45 (R1) and 47 (R2).

Figure 2 shows the top-k coverage $C(k)$ of Equation (5), i.e. where the answer sits in the probmul ranking when it is present. It is ranked first for 39 (R1) and 36 (R2) units, within the top 3 for 52 and 51, and within the top 10 for 55 and 55. Every in-pool answer is ranked within the top 20. A third attempt, if the rules allowed one, would add 3 and 6 units. The two rankers are close to redundant. They put the answer at the same rank for 51 of 56 (R1) and 48 of 55 (R2) in-pool units; the Pearson correlations of the two ranks are 0.991 and 0.961. Swapping the ranker that picks

Table 1: The six full runs (all 120 tasks trained and searched). Scores are recomputed from the submission files. For B and C († downloaded by hand as notebooks with outputs; no submission file saved), they are recomputed from the correct-unit set printed by the decoder's pass@2 benchmark. R1 and R2 log per-unit pool statistics. R1, R2 and C run the same program with the same seeds. ‡C is a run of 2026-07-10 from a notebook copy; its label "v25" in our files is a download file name, not a verified Kaggle version number.

label	notebook, version	wall (h)	units (/172)	task-norm (/120)
A	`arc-2026d`, v4	4.68	51	37.67
R1	`arc-2026d-with-codex`, v1	4.77	49	35.83
R2	`arc-2026d-with-codex`, v2	4.75	45	33.50
B	`fork-of-arc-2026d-32-22lb-updated-v3`†	4.69	51	37.83
C	`arc-2026d-with-codex`, run of 2026-07-10†‡	4.75	51	37.67
D	`experiment-with-arc-agi-2`, v10	4.80	47	33.67

Table 2: Task-clustered bootstrap 95 % intervals (20,000 resamples of the 120 tasks, seed 1), task-normalised points of 120. They describe the sampling of tasks with both runs held fixed, not run-to-run variation.

quantity	value	95 % interval
S, R1	35.83	26.67 to 45.33
S, R2	33.50	24.50 to 43.00
O, R1	40.50	31.00 to 50.33
O, R2	41.17	31.67 to 50.83
H, R1	+4.67	1.33 to 8.67
H, R2	+7.67	3.17 to 12.83
$S(R1) - S(R2)$	+2.33	−1.67 to 6.50
$O(R1) - O(R2)$	−0.67	−5.50 to 3.83

the two attempts changes the score by less than the run-to-run spread of Section 4.4. Submitting the kgmon top 2 instead of the probmul top 2 would answer 48 units (34.17, −1.67) in R1 and 47 units (34.83, +1.33) in R2; the decoders' own benchmarks print the same values. Run D evaluates three selectors on one candidate pool, scoring the two grids each would submit. Its log reports 33.67, 33.67 and 34.67 (`CURRENT`, `IMBUE_EXACT`, `IMBUE_QWEN_DIVERSE`; as printed by the run). The first value matches our re-scoring of D's submission. These comparisons are not independent tests, and none exceeds the run-to-run spread. Selection and generation are coupled: a larger pool, for example at a lower threshold, could make selection harder, so the headroom measured here does not bound the headroom of a larger pool.

4.2 Units without a correct candidate

In R1, 116 of 172 units (67.4 %) receive no correct candidate above the threshold; in R2, 117 (68.0 %). Because the threshold filters on the model's own probability, these units mix answers the model does not favour with answers it rates below 0.2; we cannot separate the two. Figure 3 splits these units by the mean cell agreement of their candidates with the answer, where a candidate whose shape differs from the answer counts as 0:

$$a_u = \frac{100}{|C_u|} \sum_{c \in C_u} \mathbf{1}\big[\mathrm{shape}(c) = \mathrm{shape}(y^\star_u)\big] \cdot \frac{1}{|y^\star_u|} \sum_{(i,j)} \mathbf{1}\big[c_{ij} = y^\star_{u,ij}\big], \tag{6}$$

Table 3: Pool coverage and selection in the two logged runs (units of 172; task-normalised score of 120 in parentheses).

	R1	R2
submitted pair correct (pass@2)	49 (35.83)	45 (33.50)
answer in candidate pool (oracle selection)	56 (40.50)	55 (41.17)
selection headroom	+7 (+4.67)	+10 (+7.67)
no correct candidate above the threshold	116 (67.4 %)	117 (68.0 %)
answer at probmul rank 1	39	36
answer at probmul rank ≤ 2 (= submitted pair)	49	45
answer at probmul rank ≤ 3	52	51
answer at probmul rank ≤ 5	54	54
answer at probmul rank ≤ 10	55	55
answer at probmul rank ≤ 20	56	55
same rank under probmul and kgmon	51 of 56	48 of 55
kgmon top 2 submitted instead	48 (34.17)	47 (34.83)

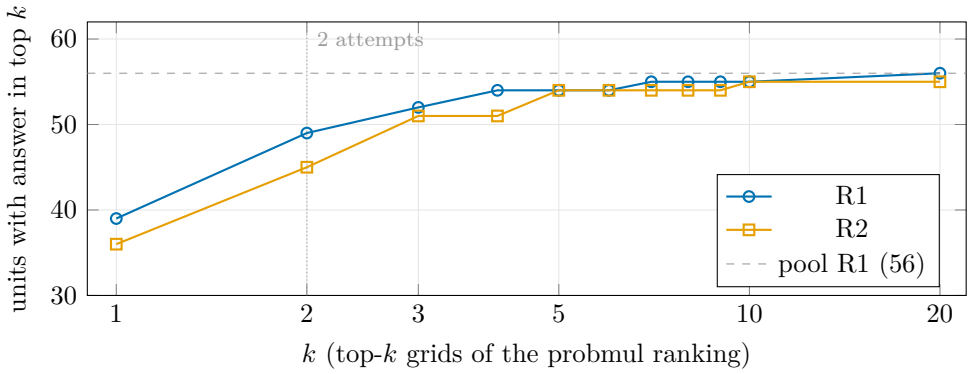


Figure 2: Top-k coverage of the answer in the two logged runs. The submitted pair equals the probmul top 2 ($k = 2$); beyond $k = 10$ almost nothing remains to select (the pool of R2 has 55 units, that of R1 56).

with $|y_u^\star|$ the number of cells of the answer (notebook cell 30 of R2). We use the cut points of the runs' own analysis: above 90 % is *close*, 50–90 % *partial*, otherwise *far*. As a baseline we also compute the agreement of the test input itself with the answer, by the same rule.

- **Close (47 in R1, 48 in R2).** The mean agreement over all candidates exceeds 90 %, yet no candidate is exact. Simply copying the test input also exceeds 90 % for 26 of the 47 (R1) and 27 of the 48 (R2); its mean agreement on these units is 90.3 % and 88.3 %. Across all 172 units, copying the input passes the 90 % bar for 39. So this class mostly sorts tasks by how similar the output is to the input, and does not by itself show that the candidates are near the answer. The candidates do beat the copy baseline on most close units: copying matches or beats them on 6 in each run.
- **Partial (32, 33)** and **far (22, 22).** Copying the input exceeds 90 % for 1 partial unit in each run and for at most 1 far unit.

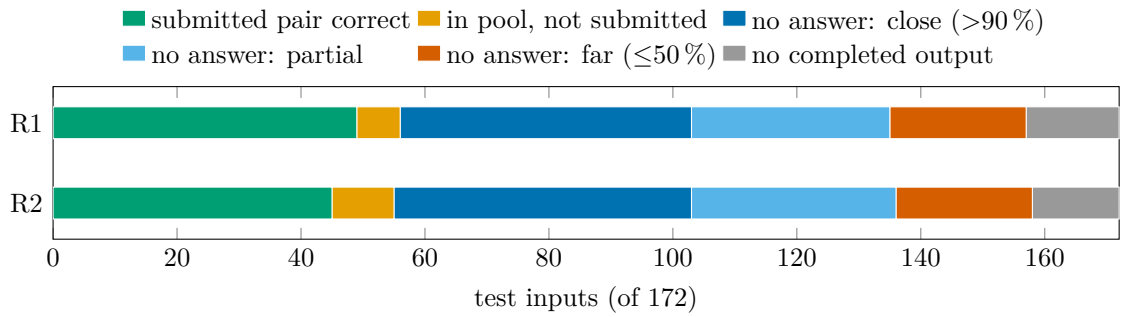


Figure 3: Where each test input ends in the two logged runs. Only the orange segment can be recovered by selection from these pools; the other unsolved segments have no correct candidate above the threshold. The close/partial/far classes use the mean cell agreement of Equation (6); for more than half of the close units, copying the test input also exceeds 90 %.

- **No completed output (15, 14).** The search returned no output above the probability threshold, the log records `FALLBACK_SAVE ... NOT_PROCESSED`, and an all-zero placeholder was submitted. These units belong to 11 tasks in each run. All of them finished normally (in 134–873 s in R1 and 182–802 s in R2). Neither run logged a task time-out or an exception. The longest logged interval after a search call starts took 227.1 s in R1 and 246.9 s in R2, below the 540 s cap.

Units whose answer is in the pool but was not submitted (7 and 10) are the only losses that a better selector could recover from these pools.

A single task illustrates the candidates of such units, and how they vary between runs. For task `142ca369`, R2's repair analysis prints the closest candidates for both test inputs (as printed by the run). They agree with the answer on 72.84 % and 93.16 % of cells, and at every one of the 88 and 26 differing cells the candidate predicts background (colour 0) where the answer has a colour. In R1, the closest candidate for the first test input agrees on 95.68 %; its 14 wrong cells have colour 2 where the answer has 6. In run C, a run of the same program, a correct grid for this task is in the pool at rank 7 under both rankers (as printed by the run; the table is per task, so we cannot say which test input). For this task, the same program loses the answer in generation in R1 and R2 and in selection in C.

4.3 Candidate diversity

A unit receives 14.4 candidates on average in R1 and 14.0 in R2, but only 7.6 and 7.3 distinct grids (these averages include the placeholder units). The duplicate ratio of a unit is

$$d_u = 1 - \frac{|P_u|}{|C_u|}, \tag{7}$$

and its mean over the 172 units, including placeholder units with $d_u = 0$, is 0.381 in R1 and 0.378 in R2. Excluding the placeholder units, the averages are 15.64 candidates and 8.19 distinct grids in R1, and 15.20 and 7.85 in R2. The largest candidate multiset has 27 candidates (at most 19 distinct grids), well below the bound of 64 of Equation (1). Figure 4 shows the distribution of distinct grids per unit. Twenty-six units in R1 and 33 in R2 have at most 3 distinct grids, and for 14 and 19 of these, none of the grids is the answer. For example, for `5961cc34` (test input 0), all 16 candidates are the same wrong grid in both runs, with 95.9 % of cells right.

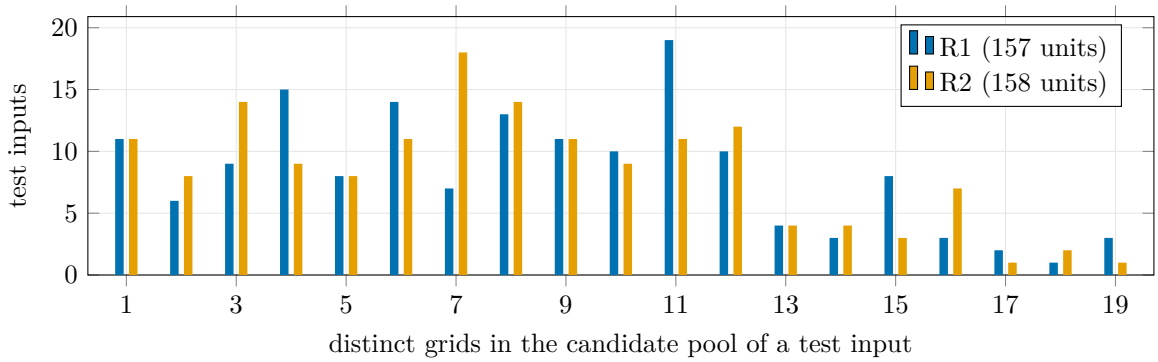


Figure 4: Distinct grids per test input, excluding the units with only a placeholder grid. Each unit is searched in up to 16 views.

Table 4: Three runs of one program with identical seeds. C has no saved submission file or log; its correct-unit set is the one printed by the run.

run	units (/172)	task-norm (/120)
R1	49	35.83
R2	45	33.50
C	51	37.67
correct in all three	41	31.00
correct in at least one	57	41.67

Duplicates are not by themselves a failure. Both rankers reward grids found in several views, so agreement between views is the signal they use. Low diversity is in fact associated with the answer being present: among units with a real candidate, the answer is in the pool for 12 of the 26 units with at most 3 distinct grids against 44 of the other 131 in R1 (46.2 vs. 33.6 %), and for 14 of 33 against 41 of 125 in R2 (42.4 vs. 32.8 %). We therefore read the distinct-grid count as a description of the pool, not as a cause of failure. The search draws no random samples and keeps only outputs with probability above 0.2, but batched inference is numerically non-deterministic [11, 16], and training losses also differ between runs with identical seeds. The views differ by symmetry, colour relabelling and demonstration order, and they often lead the adapted model to the same grid. The search has no term that rewards diverse outputs [cf. 18], and the pipeline does not draw stochastic samples. We did not measure whether either would add correct grids (Section 7).

4.4 Run-to-run variation and oracle unions

R1, R2 and C run the same program with the same seeds (Section 3). Table 4 gives their results. They answer 45 to 51 units, a spread of 6 units and 4.17 points; 41 units are correct in all three and 57 in at least one. R1 and R2 alone differ by 4 correct units in total, but they disagree on 10 units (7 correct only in R1, 3 only in R2; 42 shared, 52 in the union), and their pools disagree on 11 units (6 only in R1, 5 only in R2; 50 shared, 61 in the union, 44.50 points). Their headroom differs by 3.00 points. Exact McNemar tests find no difference between the two runs ($p = 0.34$ on submissions, $p = 1.0$ on pools). Five of C's correct units are correct in neither R1 nor R2, and four of them are in neither logged pool. So at least 65 units are in a pool or a submission of these three runs (61 + 4).

The six full runs score 33.50–37.83 (45–51 units; Table 1), a range almost entirely covered by

the three runs of one program. With $R_j = \{u : c_u = 1 \text{ in run } j\}$, the union after m runs is

$$U_m = \Big| \bigcup_{j=1}^{m} R_j \Big|. \tag{8}$$

U_m is an oracle: a real entry still submits two attempts per test input. The five runs also differ in code, so the union mixes code changes with run-to-run variation. Figure 5 shows U_m over five of the full runs (A, R1, R2, B, C, in this order): 51, 57, 58, 61 and 62 units, or 37.67, 42.00, 42.50, 44.00 and 45.00 points. Over all 120 orders of these five runs, the union after two runs lies between 52 and 59 units, and it reaches 62 after all five. The sixth full run, D, adds no new unit. Thirty-seven units (27.50 points) are correct in all five runs; of the 62 units in the union, 4 are correct in only one run. Any two of these five runs share 41–45 correct units out of a union of 52–59. Capture-recapture estimates give lower bounds on the number of units these code versions can answer: 62.7 (Chao1 over the five submissions) and 61.6 for the pools (Lincoln-Petersen on R1 and R2). Both are biased low by differences between tasks, and the 70-unit evidence union below already exceeds the pool estimate.

Repeated runs therefore do add correct candidates. The union of the five full runs (45.00) and the union of the two pools (44.50) both exceed the oracle score of either single run (40.50, 41.17). The union of the two pools exceeds the larger single pool by 3.33 points (bootstrap 95 % interval 1.00 to 6.33). The union of all 41 distinct results, including partial runs, is 64 units (46.50 points, 38.75 %), against 51 units (37.83) for the best single run. Submissions and the two logged pools together cover 70 units (50.83 points, 42.36 %). For at most 101 of the remaining 102 units, no submission and no pool we have contains the answer: run C's printed task-level table adds at least one (a test input of `142ca369`).

On the public leaderboard, ten submissions of one notebook version, each a fresh run, scored a mean of 29.36 % (sd 1.65; range 27.36–32.64 %, a difference of 5.28 points). NVARC report a variability of 1–2 points [16]. The highest, 32.64 % (39.17 of 120), is the best of the 63 submissions in our records and therefore biased upward. That version was never run on all 120 evaluation tasks and still used a randomised scoring seed. The notebook of R1 and R2 scored 27.64–28.89 % over 3 submissions. A single leaderboard score of this pipeline therefore says little about a code change of a few points.

5 Pitfalls in the notebooks' own diagnostics

The analysed notebooks print their own diagnostics. Several of them would lead to the opposite conclusion if quoted as printed. We list them because they are easy to repeat in any pipeline of this kind. Every number in this paper is recomputed, or quoted from a run log and marked as printed by the run.

1. **Generation losses reported as ranking losses.** A survival table in R2 prints "Lost at ranking stage: 117 (68.0 %)". These 117 units have no correct candidate at all; the loss happens before ranking.

2. **Inverted comparisons.** A score audit in R2 labels the answer as having the "better" search and augmentation score when its NLLs are higher (worse), e.g. augmentation mean 12.78 vs. 4.18 for the rank-1 grid.

3. **A losing variant reported as the winner, from a test of a different ranker.** The "scoring optimization" experiment in R2 prints "WINNER" and a recommendation for a

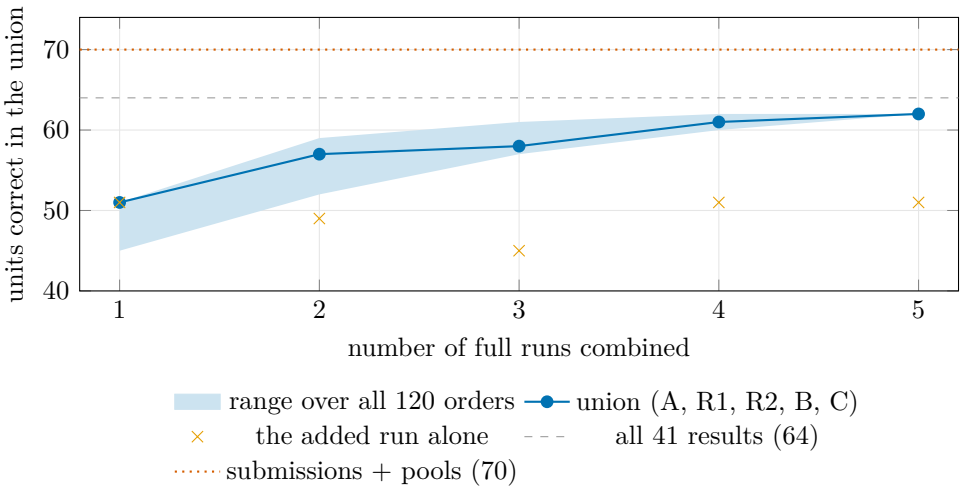


Figure 5: Oracle union of correctly answered test inputs as full runs, which differ in code, are combined. The dotted line (submissions and the two logged pools) excludes the task-level pools of B and C.

variant that is 4.67 points below the baseline. All seven variants lost (between −4.67 and −9.33, as printed). The variants are not re-weightings of the shipped ranker. The experiment scores every occurrence of a grid on its own, which drops the occurrence count that both shipped rankers reward and lets one grid fill both attempts. With a per-occurrence baseline, the search term is also constant, so four of the variants are one and the same ranking and print the same score (26.50).

4. **Placeholders reported as crashes.** Units with only the all-zero placeholder are labelled "Worker crashed or timed out". In the logs, these tasks finished normally and the search returned no completed output (Section 4.2). A second label, "Pruned by max_score", describes the outcome correctly, but the notebook derives it from the placeholder's dummy score of 999, not from the search.

5. **Simulations that cannot see what they claim to measure.** A threshold sweep only re-filters candidates that already passed the real threshold, so it cannot show pruned answers. A "beam width" sweep is top-k of the existing pool. A meta-selector simulation reports 0.00 against a 33.50 baseline.

6. **Headroom counted from the wrong baseline.** The top-k experiment prints that selection "can recover at most 18 additional puzzles" (R2; 15 in R1), counting from top 1. Two attempts are already submitted, so the gain to top 5 is 9 units (R2) and 5 (R1).

7. **Scores over the wrong denominator.** Run B prints "37.8/111", i.e. over the 111 tasks with any decoded output. The score is 37.83 of 120 (51 of 172 units).

8. **Pool coverage reported as solved.** Run B's summary "47/120 puzzles solved" counts a task if any candidate anywhere is correct (an oracle). A later cell's "34/120 puzzles solved by dual selection" counts only answers at rank 1 and ignores attempt 2. Both differ from the decoder's own pass@2 benchmark, whose correct-unit set we use for B (Table 1).

9. **Methods that do not match the code.** Run B's description says the dual selector takes the top grid of each of the two rankers. As shown in Section 2, attempt 2 is the probmul rank-2 grid whenever one exists, and kgmon never changes the submitted pair. Run B prints the same score, 37.8, for probmul alone and for the dual selector.

6 Related work

Test-time training on augmented demonstrations [1] and augmentation-based scoring of candidates were key ingredients of the strongest open ARC Prize 2024 entries [6], among them the ARChitects, whose depth-first search and product-of-experts rescoring [10] the pipeline studied here inherits. NVARC [16] carries them into the ARC-AGI-2 setting [7], adds a large synthetic dataset and a batched search, and combines the result with a Tiny Recursive Model [13]; the ARC Prize 2025 technical report [9] summarises the 2025 competition. We study only the Qwen3-4B component, as modified in our notebooks; our runs differ from NVARC's in the ranker that picks the attempts and in the other changes listed in Section 2. Outside ARC, repeated sampling studies [3] separate coverage (does any sample solve the problem?) from selection (can the solver pick it?), the same split we measure here. Diverse decoding [18] could enlarge the candidate pool; we did not test it. Numerical non-determinism of batched inference [11] is one candidate source of the run-to-run variation we measure.

7 Threats to validity

- **One set of generation settings.** All six full runs use the same 4B model, the same test-time training recipe and the same thresholded search (threshold 0.2, 16 views, 128 training samples), as modified in our notebooks (Section 2). The results describe this pipeline as it was run. They do not show that no NVARC variant can exceed these numbers.
- **Generation-side variants without pool data.** No full-run or pool measurement exists for a generation-side variant. Three of our notebooks did change generation settings (threshold 0.15, 24 views, 24 colour permutations in test-time training), but their saved runs cover at most 4 evaluation tasks. Their only full evidence is leaderboard scores (means 24.86 % and 29.54 % over 3 submissions each), too few and too noisy to conclude anything. We did not test a diversity term, stochastic sampling or a larger model in this pipeline. Runs of one program already reach a pool union of 61 units against 55 and 56 for each run alone, so more sampling could add correct grids; how many is open.
- **Threshold dependence.** "No correct candidate" and the split between generation and selection are conditional on the threshold $p > 0.2$. We cannot separate answers pruned by the threshold from answers the model does not favour, since we did not score the answer under the adapted model.
- **Training-data overlap.** The model's synthetic training data were in part derived from descriptions of the 120 public evaluation tasks, by our inference from NVARC's training script (Section 3). Pool coverage on this set may not transfer to unseen tasks. Evaluation scores of 27.92–31.53 % against a leaderboard mean of 29.36 % suggest that the effect on the score is small, but we did not measure it.

- **Run-to-run variation and code versions.** Three runs of one program span 6 units (4.17 points), so single-run numbers carry noise of that order. The other full runs also differ in code, and we cannot show that any of the runs shared a Kaggle image or GPU assignment.
- **Logged pools.** Per-unit pool statistics come from two runs of one program. A and D provide submission-level data only. B and C print task-level pool tables but no per-unit statistics.
- **Oracles.** Pool coverage and unions over runs are oracle quantities; a real entry submits two attempts per test input.
- **Statistical uncertainty.** The intervals of Table 2 resample tasks with the runs held fixed; they do not include run-to-run variation.
- **Quoted diagnostics.** Run D's three-selector comparison, run C's rank for `142ca369` and the `142ca369` cell counts are quoted from the run outputs; the candidate grids were not saved, so we could not recompute them. D's baseline value matches our re-scoring. The rank correlations and duplicate ratios we recomputed from the per-unit tables.
- **Public evaluation set.** All pool and union statistics are on the public evaluation set; for the hidden tasks we only have leaderboard scores.
- **Coarse closeness measure.** Mean cell agreement over all candidates ignores which cells are wrong and penalises shape errors fully. For more than half of the close units, copying the test input also exceeds 90 %. A measure restricted to the cells where the answer differs from the input needs the candidate grids, which were not saved.

8 Conclusion

On the public ARC-AGI-2 evaluation set, at the shared settings (threshold 0.2, 16 views), the NVARC-derived pipeline as we ran it loses most test inputs before selection: 116 and 117 of 172 receive no correct candidate above the threshold in the two runs whose pools we logged. Perfect selection would add +4.67 and +7.67 task-normalised points, 13.0 % and 22.9 % of the score. For more than half of the test inputs without a correct candidate whose candidates exceed 90 % mean cell agreement, copying the test input also exceeds 90 %. Three runs of one program with identical seeds answer 45 to 51 test inputs, and coverage grows from 55–56 units in one pool to 61 in two pools and 70 in all our submissions and pools together. Resubmissions of one notebook version vary by up to 5.28 percentage points on the leaderboard (sd 1.65). At these settings, most lost test inputs have no correct candidate, so generation-side changes are where most of the remaining headroom would have to come from; we did not measure such changes. Selection leaves 4.67 to 7.67 points.

Code and data. The analysis scripts (`scripts/score_all.py`, `scripts/paper_stats.py`) and the per-run tables are available from the authors on request. This paper is attached to our ARC Prize 2026 Paper Track writeup on Kaggle.

Use of AI tools. AI coding and writing assistants were used to write analysis scripts and to draft the text. All numbers were recomputed from saved run outputs and checked by the authors, who take full responsibility for the content.

References

[1] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. *arXiv preprint arXiv:2411.07279*, 2024.

[2] ARC Prize Foundation. ARC-AGI-2 public training and evaluation data. `https://github.com/arcprize/ARC-AGI-2`, 2025.

[3] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[4] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[6] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical Report. *arXiv preprint arXiv:2412.04604*, 2024.

[7] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[8] François Chollet, Mike Knoop, Greg Kamradt, Walter Reade, María Cruz, and Addison Howard. ARC Prize 2026 - ARC-AGI-2. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2`, 2026. Kaggle.

[9] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical Report. *arXiv preprint arXiv:2601.10904*, 2026.

[10] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. *arXiv preprint arXiv:2505.07859*, 2025.

[11] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`, 2025. Thinking Machines Lab: Connectionism.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[13] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[14] Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.

[15] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[16] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025. `https://github.com/1ytic/NVARC`, 2025. Write-up `nvarc_2025.pdf` in the repository. Model: `https://www.kaggle.com/models/sorokin/qwen3_4b_grids15_sft139`.

[17] Unsloth AI. Unsloth. `https://github.com/unslothai/unsloth`, 2024.

[18] Ashwin K. Vijayakumar, Michael Cogswell, Ramprasaath R. Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424*, 2016.